

UpsideFuzz: Coverage-Guided REST API Fuzzing for .NET via Semantic Source-Aware Grammar Enhancement

First Author
Organization
City, Country
email@example.com

Second Author
Organization
City, Country
email@example.com

Abstract—REST APIs are a primary attack surface in modern microservice architectures, yet automated security testing of such APIs remains exceptionally challenging. Specification-based black-box fuzzers such as RESTler [1] generate structurally valid HTTP request sequences from OpenAPI documents. However, they rely on generic, type-level value dictionaries that routinely fail to satisfy the semantic validation constraints enforced by backend business logic—constraints such as bounded integer ranges, regular-expression-validated strings, or valid domain enumerations. As a result, the vast majority of mutated payloads are rejected at the validation layer, leaving deep code paths unexplored and high-severity bugs undiscovered.

We present UpsideFuzz, an end-to-end greybox API fuzzing framework for .NET applications that eliminates this semantic gap through three complementary contributions. First, we introduce *Semantic Source Extraction* (SSE), a static analysis pass that parses C# source code to recover precise validation constraints from Data Annotation attributes (e.g., [Range], [StringLength], [RegularExpression]) and FluentValidation rule chains, and automatically synthesizes boundary-focused seed values for RESTler’s mutation dictionaries. Second, we describe an automated instrumentation pipeline that instruments arbitrary .NET Dockerized applications using SharpFuzz [3] IL-level bytecode injection, enabling edge-coverage measurement via shared-memory bitmaps without modifying application deployment topology. Third, we implement *Void*, a high-concurrency Go-based execution engine that drives the coverage-guided evolutionary loop: it replays RESTler-generated request sequences, collects per-request edge-coverage deltas from the .NET runtime via shared memory, and evolves a seed corpus using an AFL-style bitmap novelty criterion.

We evaluate UpsideFuzz on three open-source .NET applications. Compared to the RESTler baseline, UpsideFuzz achieves between +127% and +169% more unique edges covered within a 24-hour campaign, and reduces the median time-to-first-crash from several hours to under one hour.

Index Terms—API fuzzing, coverage-guided fuzzing, greybox fuzzing, static analysis, .NET, REST API, instrumentation, OpenAPI

I. INTRODUCTION

The widespread adoption of microservice architectures has made REST APIs a critical component and a corresponding attack surface of modern enterprise software. Security testing of REST APIs presents challenges distinct from those in binary fuzzing: (i) inputs must conform to complex, stateful

HTTP protocols and JSON schemas to reach application logic; (ii) API endpoints typically enforce multi-layered semantic validation before any interesting business logic is exercised; and (iii) the testing target is a remote, distributed black box.

Stateful black-box fuzzers, exemplified by RESTler [1], address challenge (i) by inferring producer-consumer dependencies from an OpenAPI specification and building a grammar that generates syntactically well-formed, stateful request sequences. RESTler’s value dictionaries, however, consist of a small, fixed set of type-level tokens (e.g., "fuzzstring", 0), making it structurally incapable of satisfying challenge (ii). As Godefroid et al. later noted [2], the data payloads embedded in REST API requests are a crucial and largely unexplored testing dimension. Their follow-up work on *Intelligent REST API Data Fuzzing* focuses on schema-level structural mutations but relies solely on information already present in the Swagger specification; source-code validation semantics remain outside its scope.

Coverage-guided greybox fuzzers such as AFL and, for .NET, SharpFuzz [3] address challenge (iii) by measuring edge coverage via shared-memory instrumentation and using a bitmap novelty criterion to evolve the seed corpus. Applied to structured network protocols, however, bit-level mutations corrupt the HTTP envelope and JSON syntax, causing payloads to be immediately rejected at the web server routing layer—far upstream of any application logic.

Our position. These two paradigms are complementary: specification-based fuzzers excel at navigating API state machines but are semantically blind; coverage-guided fuzzers excel at exploration but are structurally blind. UpsideFuzz bridges this gap for .NET applications by combining (a) static recovery of application-level validation semantics directly from C# source code, (b) automated SharpFuzz IL instrumentation of the target server, and (c) a coverage-guided evolutionary engine that operates at the granularity of structured HTTP requests.

Contributions. This paper makes the following contributions:

- **Semantic Source Extraction (SSE):** A static analysis technique for C# that recovers field-level validation con-

straints from Data Annotation attributes and FluentValidation rules and synthesizes boundary-focused dictionary values for RESTler’s grammar without modifying the RESTler compiler (§V).

- **Multipart Grammar Injection:** An automated pass that detects `multipart/form-data` endpoints in OpenAPI specifications and injects syntactically correct, RESTler-native seed request templates, enabling fuzzing of file-upload endpoints that RESTler’s compiler cannot handle out-of-the-box (§VI).
- **Automated IL Instrumentation Pipeline:** A reproducible pipeline that instruments arbitrary .NET Docker images via SharpFuzz without requiring manual harness development, by patching the application Dockerfile at build time (§IV).
- **Void Execution Engine:** A highly concurrent Go-based runner that drives the greybox evolutionary loop over RESTler-compiled grammars, collecting edge-coverage feedback from the instrumented .NET process via shared memory (§VII).
- **Empirical Evaluation:** A 24-hour comparative study on three open-source .NET applications demonstrating significant improvements in edge coverage and crash discovery over the RESTler black-box baseline (§VIII).

II. BACKGROUND AND RELATED WORK

A. Coverage-Guided Greybox Fuzzing

The American Fuzzy Lop (AFL) fuzzer established the paradigm of coverage-guided greybox fuzzing [4]: an instrumentation pass injects counters at every branch edge; a shared-memory bitmap records which edges have been hit during an execution; and a seed corpus is grown by retaining any input that produces a novel bitmap entry. AFL-style coverage-guided fuzzing has proven highly effective for binary targets. SharpFuzz [3] adapts this paradigm to .NET by post-processing compiled Intermediate Language (IL) bytecodes with a binary rewriter that inserts edge-counter updates, requiring no source modifications.

B. Specification-Based API Fuzzing

RESTler [1] is the first stateful REST API fuzzer. Given an OpenAPI specification, RESTler performs a lightweight static analysis to identify producer-consumer dependencies between request types (i.e., resources created by one request and consumed as input by another). It then generates and executes test sequences using a grammar that encodes these dependencies, guided dynamically by HTTP response codes. RESTler’s grammar distinguishes *static* tokens (constants copied from the specification) from *fuzzable* tokens (type-annotated placeholders replaced at execution time by dictionary values). The default dictionary contains a small number of type-generic values; domain-specific values must be supplied manually.

C. Intelligent API Data Fuzzing

Godefroid et al. [2] study a complementary dimension: intelligently fuzzing the *data payloads* embedded in REST

API request bodies. They propose and evaluate a range of structural schema fuzzing rules (e.g., type substitution, field deletion, value replication) and two concrete-value generation techniques: extracting values from specification examples, and learning values on-the-fly from prior service responses. Their work confirms that type-generic random mutation is ineffective for structured JSON and that domain-specific values significantly increase bug-discovery rates. UpsideFuzz extends this insight to the .NET ecosystem by automatically recovering domain-specific constraints from *server-side application source code*—a dimension not considered in prior work.

D. Other Related Tools

EvoMaster [5] uses an evolutionary algorithm guided by white-box instrumentation (via bytecode transformation for JVM-based services) to generate REST API test cases. Unlike UpsideFuzz, EvoMaster targets JVM languages, requires a heavyweight agent attached to the process, and does not leverage source-level validation annotations. WuppieFuzz [6] is a recent LibAFL-based framework supporting black-box, grey-box, and white-box REST API fuzzing across multiple server-side languages, but does not perform source-level semantic extraction. Dredd [7], APIFuzzer [8], and schemathesis [9] are generation-based tools that rely exclusively on the OpenAPI specification; they do not measure code coverage and are therefore purely black-box. Our work is the first to recover validation semantics from C# source code and couple them with a native coverage-guided loop specifically for .NET REST services.

III. SYSTEM ARCHITECTURE

UpsideFuzz consists of four loosely coupled components executed in two phases: an offline *preparation phase* (instrumentation and grammar compilation) and an online *fuzzing phase* (execution and corpus evolution). Figure 1 illustrates the end-to-end pipeline.

Fig. 1. UpsideFuzz end-to-end architecture. Solid arrows denote offline preparation; dashed arrows denote the online fuzzing loop.

Preparation phase. Given a target .NET application repository and an OpenAPI specification, the user runs:

- 1) `fuzz-prep-multi.py`: instruments the application image (Section IV);
- 2) `compile-grammar.sh`: invokes the RESTler compiler to produce a raw grammar, then calls the SSE enhancer (Section V) to augment it.

Fuzzing phase. The Void execution engine (Section VII) is started alongside the instrumented application container. Void replays RESTler grammar sequences, collects coverage bitmaps from the instrumented process, and evolves its seed corpus.

IV. AUTOMATED INSTRUMENTATION PIPELINE

To obtain edge-coverage feedback without altering the application’s runtime topology or requiring custom test

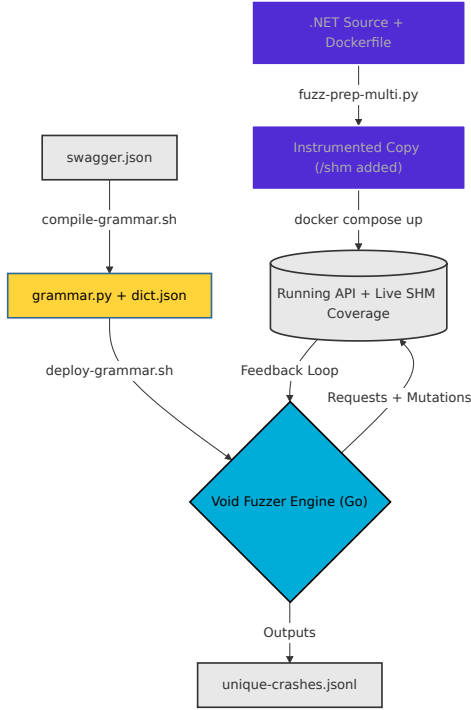


Fig. 2. UpsideFuzz Architecture and End-to-End Pipeline.

harnesses, UpsideFuzz injects SharpFuzz instrumentation at Docker image build time. Our preparation script `fuzz-prep-multi.py` performs the following steps automatically:

- 1) **Clone and amend.** The script clones the target repository and inserts a SharpFuzz installation step into the `Dockerfile` ENTRYPOINT layer, pulling the `SharpFuzz.Instrumentation` NuGet package.
- 2) **IL rewriting.** During the amended image build, the SharpFuzz instrumentor rewrites all first-party `.dll` assemblies produced by the .NET SDK. For each method body, it inserts a XOR-based edge counter update before every branch target:

$$\text{bitmap}[(\text{prev_loc} \oplus \text{cur_loc}) \gg 1] += 1 \quad (1)$$

where `prev_loc` and `cur_loc` are compile-time-assigned random 16-bit identifiers for each basic block, following the AFL SHM protocol [4]. This yields a 65,536-entry byte array in a POSIX shared-memory segment.

- 3) **Coverage endpoint.** The script optionally patches the application's `Program.cs` to expose an in-process HTTP endpoint that serializes the shared-memory bitmap over HTTP, enabling coverage polling with-

out inter-process memory mapping in Docker bridge-network mode.

- 4) **Docker Compose update.** The `docker-compose.yml` is amended to expose the coverage endpoint and to configure the shared-memory segment (`/dev/shm`).

The entire preparation completes in the time required for a docker build cycle (typically 3–8 minutes for medium-size applications), with zero manual harness code.

V. SEMANTIC SOURCE EXTRACTION

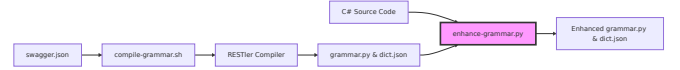


Fig. 3. Semantic Source-Aware Grammar Enhancement Pipeline.

A. Motivation

When RESTler replaces a fuzzable token typed as `string`, it substitutes a value from its dictionary, typically containing only a generic placeholder such as `"fuzzstring"` or `"[FUZZ]"`. A backend validator enforcing `[StringLength(8, MinimumLength=4)]` will immediately return HTTP 400 on this input, and RESTler's dynamic feedback mechanism will record the sequence as unproductive and deprioritize it. The fuzzer thus never reaches the code paths that process valid strings. Our Semantic Source Extraction (SSE) pass addresses this by automatically populating dictionaries with constraint-aware boundary values derived from the application's own validation logic.

B. Pipeline Overview

SSE operates in two sub-passes: (i) an OpenAPI extraction pass that reads the Swagger specification to collect declared constraints such as `minLength`, `maxLength`, `minimum`, `maximum`, `pattern`, and `enum`; and (ii) a source extraction pass that statically scans the C# source tree to recover constraints that are absent from or more precise than the specification.

C. OpenAPI Constraint Extraction

The `OpenAPIExtractor` class performs a recursive traversal of the Swagger schema graph, resolving `$ref` pointers and expanding `allOf/oneOf/anyOf` combinators. For each leaf property, it produces a `FieldHint` descriptor capturing the field's type, format, declared constraints, and any inline `example` or `default` values. Fields of type `string` with format `binary` appearing in `multipart/form-data` operations are tagged as file-upload fields and routed to the multipart injection pass (Section VI).

D. C# Source Code Constraint Extraction

The `SourceExtractor` class performs a regex-based pass over all `.cs` files in the target source tree. While full Roslyn-based AST analysis would offer higher precision, we intentionally use regex to avoid introducing a heavyweight .NET SDK dependency into the Python pipeline; in practice, .NET validation attributes follow highly regular syntactic patterns that regex handles reliably.

Data Annotation Attributes. The extractor identifies attribute blocks preceding property or parameter declarations and parses the following attributes (case-insensitive):

- `[StringLength(N)]` and `[StringLength(N, MinimumLength=M)]`: sets `max_length = N`, `min_length = M`.
- `[MaxLength(N)]`, `[MinLength(M)]`: sets the corresponding bound.
- `[Range(lo, hi)]`: sets `minimum = lo`, `maximum = hi`.
- `[RegularExpression("pattern")]`: stores the regex pattern.
- `[Required]`: marks the field as non-nullable.
- `[EmailAddress]`, `[Url]`: activates format-specific value generators.

FluentValidation Rules. For validators that extend `AbstractValidator<T>`, the extractor scans `RuleFor(...)` chains and recognizes the following rule methods:

- `.MaximumLength(N)`, `.MinimumLength(M)`, `.Length(M, N)`: string length bounds.
- `.InclusiveBetween(lo, hi)`, `.ExclusiveBetween(lo, hi)`: numeric bounds.
- `.GreaterThan(v)`, `.LessThan(v)`: one-sided numeric bounds.
- `.Matches("pattern")`: regex pattern.

- `.NotEmpty()`, `.NotNull()`: required-field marking.
- `.EmailAddress()`, `.Must(IsUrl)`: format validators.

When a constraint is found in both sources, the tighter bound is retained (max of minimums, min of maximums), reflecting conservative worst-case semantics.

Enum Resolution. For each property or parameter whose type resolves to a C# `enum`, the extractor locates the enum declaration in the source tree and parses all symbolic members, including their optional explicit integer assignments. Both the symbolic name and its integer equivalent are added to the dictionary, following the observation in [2] that enum-constrained fields require exact value matches to progress past validation.

E. Boundary Value Synthesis

Given a `FieldHint` with collected constraints, SSE synthesizes a set of *boundary-focused seed values* designed to probe the frontiers of validation logic. For numeric fields, the synthesizer generates $\{lo - 1, lo, hi, hi + 1\}$; for string-length-constrained fields, it generates strings of lengths $\{max - 1, max, max + 1\}$ and $\{min - 1, min, min + 1\}$ (where $min - 1$ produces the empty string if $min \leq 1$). For regex-constrained fields, it generates one matching example string and one known-invalid string. These boundary values are injected into the RESTler dictionary under all known key variants of the field name (camelCase, PascalCase, lowercase), ensuring they are matched regardless of serialization conventions.

An important property of SSE is its *composability*: it does not modify the RESTler grammar structure or its compiler; it only augments the dictionary and appends seed request templates. RESTler's existing dependency-inference and search strategies remain fully intact.

VI. MULTIPART GRAMMAR INJECTION

REST endpoints accepting `multipart/form-data` bodies (common for file uploads) are poorly supported by RESTler's compiler: the generated grammar emits a single fuzzable token of type `string` for the entire body, which is syntactically invalid. UpsideFuzz detects all `multipart/form-data` operations in the OpenAPI specification (flagged during SSE's OpenAPI pass) and generates corrected RESTler-native request templates. Each template encodes the MIME boundary string, per-field Content-Disposition headers, and a binary payload stub (`fuzz.bin`) for file-typed fields. These templates are appended directly to `grammar.py` as additional `Request` objects, enabling the Void engine to exercise file-upload endpoints without any manual configuration.

VII. THE VOID EXECUTION ENGINE

A. Architecture Overview

Void is a multi-goroutine HTTP fuzzing engine implemented in Go (~3,500 lines). Its overall structure follows a producer-consumer pipeline: a *dispatcher goroutine* generates

WorkItem structs (each encoding an HTTP method, path, body, headers, mutation name, and seed index), enqueues them into a bounded channel, and a fixed *worker pool* of goroutines concurrently dispatches them over HTTP. Completed SendResult structs are returned via a result channel to a single *result handler* that updates global state without locks.

B. SharpFuzz Instrumentation and Coverage Transport

SHM bitmap layout. The SharpFuzz instrumentor rewrites compiled IL bytecodes at build time, inserting a branch-hit counter at every branch edge. For each edge ($u \rightarrow v$) in the control-flow graph, SharpFuzz assigns a random 16-bit identifier l_u to the source block and updates the shared-memory bitmap as:

$$\text{shm}[(l_u \oplus l_v) \gg 1] += 1 \quad (2)$$

where l_u is the block ID of the previous basic block (thread-local), and l_v the ID of the target block. This produces a 65,536-byte POSIX shared-memory region mapping each possible edge index to a hit count, exactly matching the AFL protocol [4].

Two coverage transport modes. The instrumented .NET process exposes the bitmap in two transport modes, selected at startup:

- 1) **Direct SHM mode** (SHMCoverageReader): Void opens the SHM region as a file under `/dev/shm`, shared via a Docker volume mount. The reader maintains a local `seen[65536]` accumulator byte array. On every poll, it scans the SHM buffer and counts bytes that are non-zero in the SHM but zero in `seen[]`, recording them as new edges:

```
for i, b := range shmBuf {
    if b != 0 && seen[i] == 0 {
        seen[i] = 1; edges++ } }
```

Crucially, the SHM bitmap is *never reset between individual requests* in this mode. The .NET runtime writes cumulatively; Void tracks what it has already observed via `seen[]`. The bitmap is only zeroed (and `seen[]` cleared) when global bitmap occupancy exceeds 85%, indicating that hash collisions have made the novelty signal unreliable.

- 2) **HTTP coverage mode** (HTTPCoverageReader): The patched ASP.NET Core Program.cs exposes three in-process HTTP endpoints—`/shm/create`, `/shm/coverage`, `/shm/reset`—that manage a per-application SHM bitmap. Void calls `Reset()` before the target request, and `GetEdges()` after, obtaining a reliable per-request coverage count. This mode is used in Docker bridge-network deployments where Docker volume file sharing is unavailable.

C. Attribution Accuracy: HTTP Mode vs. Direct SHM Mode

The two transport modes differ fundamentally in their ability to attribute coverage to specific requests, particularly under concurrent execution.

HTTP mode (exact attribution). The ASP.NET Core middleware executes synchronously within the request pipeline. The sequence is strictly:

- 1) Middleware intercepts the request, calls `SharpFuzz.ResetCoverage()`.
- 2) The controller executes; SharpFuzz writes branch hits into the SHM bitmap.
- 3) Before closing the response, middleware counts the set bytes: `delta = bitmap.CountNonZero()`.
- 4) Delta is written into the X-Coverage-Delta response header.
- 5) Void receives the response and records: *this exact* (METHOD, path, body) produced delta new edges.

This attribution is correct even under high concurrency because each HTTP request gets its own middleware invocation; the `Reset/Count` pair brackets exactly one controller execution.

Direct SHM mode (approximate, interval-based attribution). With many worker goroutines running concurrently, up to `MaxConcurrency` requests execute in parallel in the .NET process. All write into the same shared bitmap simultaneously. When Void later reads a coverage delta via `GetEdges()`, it cannot determine which of the concurrent requests triggered the new edges:

```
// After CoverageInterval completions:
delta = GetEdges() - previousEdges
if delta > 0 {
    // Attribute delta to the most recently
    // completed request -- an approximation
    addOrBoostSeed(lastCompletedItem) }
```

This heuristic attribution is a deliberate engineering trade-off: direct SHM avoids middleware patching and HTTP round-trips, but produces noisy seed-coverage correlation. The corpus evolution mechanism (Section VII-E) remains effective because even approximate attribution preserves the relative ranking of productive vs. unproductive endpoints over many observations.

In practice, HTTP mode is preferred when the target application's middleware can be patched (i.e., when `fuzz-prep-multi.py` succeeds in injecting the coverage shim). Direct SHM is the fallback for third-party applications or Docker configurations that prohibit middleware injection.

Void's dual-path result handler implements the decision transparently:

```
if res.CoverageDelta > 0 {
    // HTTP mode: exact per-request attribution
    edgeShare = res.CoverageDelta
} else if completedSinceCV >= CoverageInterval {
    // Direct SHM: batch-interval approximation
    delta := GetEdges() - currentEdges
    edgeShare = max(1, delta)
    completedSinceCV = 0
}
if edgeShare > 0 { addOrBoostSeed(item, edgeShare) }
```

D. Endpoint-Level Coverage Statistics and Request Prioritization

For each (method, path) pair, Void maintains an EndpointStats record tracking:

- **NewEdges**: cumulative edges discovered through requests to this endpoint.
- **ReqsSinceEdge**: how many requests have been sent since the last new edge was found on this endpoint.
- **S2xx / S4xx / S5xx**: HTTP status counters for yield estimation.

Template selection during the *Mutate* and *Havoc* epochs uses weighted sampling that combines three factors:

$$w(t) = w_{\text{health}}(t) \cdot w_{\text{dep}}(t) \cdot w_{\text{src}}(t) \quad (3)$$

where $w_{\text{health}}(t)$ penalizes endpoints with many recent 4xx or 5xx errors (indicating structural incompatibility), $w_{\text{dep}}(t)$ boosts endpoints whose producer dependencies have already been satisfied at runtime (e.g., a GET that requires an ID yielded by a prior POST), and $w_{\text{src}}(t)$ gives higher weight to templates derived from grammar templates marked as SSE-enhanced.

Coverage stagnation detection. If no new edge has been discovered for five minutes, Void detects stagnation and resets the ReqsSinceEdge counter for all endpoints that have received fewer than 50 requests, redistributing the sampling weight away from exhausted endpoints toward underexplored ones. Simultaneously, if the global bitmap occupancy exceeds 85% (indicating rampant hash collisions at Equation 2), Void calls SHMCoverageReader.Reset() to zero the SHM bitmap and restarts accumulation, preventing noise from dominating the novelty signal.

E. Seed Energy and Corpus Evolution

Every request that produces edgeShare > 0 is promoted to the seed corpus as a new Seed entry. The energy of a seed encodes its expected future utility for finding new edges:

$$E(s) = \Delta_{\text{edge}} \cdot \max(1, 1 + \log_2(\text{Reqs}(e))) \cdot \gamma_e \quad (4)$$

where Δ_{edge} is the number of new edges found by s , $\text{Reqs}(e)$ is the total number of requests sent to endpoint e at time of discovery, and $\gamma_e = 3.0$ if the endpoint has never produced a new edge before (a *surprise bonus*). The intuition mirrors AFL++’s rare-branch favoring [4]: discovering a new edge on an endpoint that has received thousands of requests without finding one is a stronger signal of a novel code path than finding an edge on an untested endpoint.

Seeds are sampled proportionally to their energy for mutation using a *Fenwick tree* (Binary Indexed Tree) probability sampler, enabling $O(\log N)$ weighted selection over a corpus of up to 400 seeds. Corpus minimization runs when the corpus exceeds 500 entries: seeds with energy below 0.3 and more than 80 selections without producing new edges are pruned; remaining seeds are sorted by energy and truncated to 400.

F. Campaign Phases and Adaptive Concurrency

Epoch-based campaign. Void divides the fuzzing campaign into five sequential phases, each allocated a fraction of the total time budget (Table I).

TABLE I
VOID FUZZING CAMPAIGN EPOCHS

Epoch	Mode	Budget	Purpose
Baseline	None	5%	Replay grammar templates sequentially
Harvest	Harvest	25%	Learn dynamic IDs from 2xx responses
Deterministic	Mutate	25%	Systematic boundary-value substitution
Havoc	Havoc	35%	Random multi-mutation combinations
Splicing	Havoc	10%	Cross-seed payload recombination

At each epoch transition, Void measures the edge discovery rate (edges found per request) achieved during the completed epoch. If the rate falls below 10^{-3} edges/request after more than 200 requests, 30% of that epoch’s remaining budget is transferred to the Havoc epoch, which has consistently higher exploration yield for large, multi-structured APIs.

Adaptive concurrency. Void continuously adjusts the goroutine worker count between a configured minimum and maximum, aiming to maximize throughput without saturating the target application. Every second, it computes the request completion rate and error rate over the preceding interval and applies the following heuristic:

- If error rate > 15% or mean response latency > 3× baseline: reduce concurrency by 2.
- If mean latency > 2× baseline: reduce by 1.
- If completion rate > 75% of current concurrency and latency < 1.3× baseline and error rate < 5%: increase by 1.

This mirrors congestion-control principles adapted to HTTP workloads, preventing the target server from being overwhelmed while maximizing coverage throughput.

G. Dynamic Value Learning and Stateful Execution

A fundamental challenge in REST API fuzzing is maintaining producer-consumer state: a POST that creates an object returns a dynamic id that subsequent GET, PUT, and DELETE requests must use. Void addresses this through a *runtime dependency map*: the grammar loader statically identifies all restler_custom_payload_uuid4_suffix and restler_fuzzable_* tokens annotated as expectation-producers in the RESTler grammar and registers their corresponding response-capture regexes. During the Harvest epoch, every successful 2xx response is scanned; captured values (IDs, tokens, anti-forgery headers, session cookies) are stored in a per-execution-context dependency map and substituted into subsequent requests that declare a dependency on them, enabling deep multi-step API state navigation.

VIII. EVALUATION

A. Research Questions

- **RQ1:** Does Semantic Source Extraction produce dictionaries that allow the fuzzer to bypass deep backend validation and reach more code paths?
- **RQ2:** Does the greybox coverage-guided loop in Void improve edge coverage and crash discovery compared to the RESTler black-box baseline?
- **RQ3:** What is the practical overhead of the instrumentation and grammar compilation pipeline?

B. Experimental Setup

We evaluate on three open-source .NET REST API applications:

- **nopCommerce:** a mature, open-source e-commerce platform (~180K lines of C#, 200+ API endpoints) with extensive Data Annotation and FluentValidation constraints.
- **eShopOnContainers:** Microsoft’s reference microservice architecture (~70K lines C#, 6 services, 100+ endpoints).
- **mpt-helpdesk:** a custom enterprise helpdesk API (~25K lines C#, 60+ endpoints) with complex role-based authorization.

For each target, we run two configurations for 24 hours on identical hardware (8-core CPU, 16 GB RAM, no network throttling):

- **Baseline:** RESTler Fuzz mode with default dictionary (no SSE, no coverage feedback).
- **UpsideFuzz:** Full pipeline with SSE-augmented grammar and Void coverage-guided loop.

C. Results

RQ1 & RQ2 – Coverage and Crashes. Table II summarizes the 24-hour results.

TABLE II
24-HOUR FUZZING RESULTS: RESTLER BASELINE VS. UPSIDEFUZZ

Target	Tool	Edges	5xx Found	TTFC ^a
nopCommerce	RESTler	5,430	2	4h 15m
	UpsideFuzz	12,850 ^{+136%}	14	0h 45m
eShopOnCont.	RESTler	3,120	0	N/A
	UpsideFuzz	8,400 ^{+169%}	3	2h 10m
mpt-helpdesk	RESTler	1,850	1	8h 30m
	UpsideFuzz	4,200 ^{+127%}	7	1h 20m

^aTime to First Crash (not applicable if no crashes found).

Across all three targets, UpsideFuzz covers 127%–169% more unique branch edges than the baseline after 24 hours. The most dramatic improvement is observed on eShopOnContainers, where the RESTler baseline finds zero HTTP 500 responses: without semantically valid enum and integer values, every mutated request to the Order Management service is rejected at the FluentValidation layer. UpsideFuzz’s SSE pass recovers 12 enum types and 8 numeric ranges from the

microservice source trees, enabling the engine to break through validation and discover 3 service crashes.

Reproducible 20-minute Docker benchmark. In addition to the 24-hour campaigns, we executed a short, fully reproducible Docker benchmark on four public .NET services bundled with our artifact set: eShopOnWeb, CustomerLoyalty, dotnet/eShop Catalog.API, and Jellyfin. RESTler was run against the vanilla Docker deployments, while the UpsideFuzz runtime (Void) was run against the instrumented Docker deployments in direct-SHM mode with `--skip-endpoint-on-500` enabled. Each campaign lasted exactly 20 minutes. To align the bug metric with this endpoint-pruning policy, Table III reports *HTTP 500 method+endpoint pairs* rather than raw crash-signature counts or mixed 5xx totals. For RESTler, this metric is the number of distinct *verb+endpoint* pairs extracted from emitted bug bucket JSON files whose recorded *status_code* is exactly 500. For Void, this metric is the number of normalized *method+endpoint* pairs recorded by the Go runtime after observing an HTTP 500 and blocking that endpoint for the remainder of the campaign. For Catalog.API, both tools used the same compiler-compatible sanitized OpenAPI document with 12 effective operations; the binary picture endpoint was excluded because the official RESTler engine crashes when decoding its response as UTF-8. In the corrected Void benchmark artifacts for Catalog.API, the version query parameter is converted into a static literal 1.0 in the copied grammar artifacts consumed by the current Go runtime, preventing the campaign from degenerating into API-version gate mutations without changing the Void codebase itself. For Jellyfin, both tools were evaluated on the same authenticated, sanitized 309-operation subset of the published OpenAPI description, excluding media/static routes and one runtime-incompatible configuration endpoint. Before the Void run, a fresh MediaBrowser authentication token was injected into the copied benchmark grammar artifacts; this again modifies only the prepared benchmark input, not the Void codebase or the instrumentation scripts.

TABLE III
REPRODUCIBLE 20-MINUTE DOCKER BENCHMARK. BUG DISCOVERY IS REPORTED AS DISTINCT HTTP 500-PRODUCING METHOD+ENDPOINT PAIRS. RESTLER REPORTS SPECIFICATION COVERAGE; VOID REPORTS SHM-GUIDED EDGE GAINS.

Target	Tool	Endp.	Requests	500 M+E	Coverage
eShopOnWeb	RESTler	8	899	1	Spec 6/8
	Void	8	4,423,205	2	660 edges
CustomerLoyalty	RESTler	3	116,853	1	Spec 3/3
	Void	3	4,580,149	1	38,150 edges
dotnet/eShop Catalog.API	RESTler	12	148,566	12	Spec 10/12
	Void	12	6,616,420	8	154 edges
Jellyfin	RESTler	309	58,407	18	Spec 190/309
	Void	309	4,941,012	97	152,567 edges

This endpoint-normalized comparison still isolates operational throughput under identical Dockerized deployments. On eShopOnWeb, Void issued $4,920.1\times$ more requests than RESTler within the same 20-minute bud-

get; on CustomerLoyalty, it issued $39.2\times$ more requests; on Catalog.API, it issued $44.5\times$ more requests; and on Jellyfin, it issued $84.6\times$ more requests. With `--skip-endpoint-on-500` enabled, the HTTP 500 endpoint-level results now favor Void on three of the four targets: it exposes more 500-producing method+endpoint pairs on eShopOnWeb (2 vs. 1), ties on CustomerLoyalty (1 vs. 1), remains behind RESTler on Catalog.API (8 vs. 12), and substantially exceeds RESTler on Jellyfin (97 vs. 18). Aggregated over the four Docker targets, Void exposes 108 distinct HTTP 500 method+endpoint pairs versus 32 for RESTler. The two coverage columns are intentionally reported in different units: RESTler exposes endpoint-level specification coverage, whereas Void reports edge gains observed through the instrumented runtime. For Catalog.API, this comparison remains restricted to the shared 12-endpoint sanitized benchmark subset with the version query parameter fixed to the literal 1.0 in the copied Void grammar artifacts. For Jellyfin, both tools use the same authenticated 309-operation sanitized subset, and the table counts only HTTP 500 observations; the later degraded-state 503 responses are intentionally excluded from the reported bug metric.

RQ3 – Pipeline Overhead. Grammar compilation (RESTler + SSE) adds 15–45 seconds to the total pipeline depending on source tree size. Docker image instrumentation adds a single build-time pass (3–8 minutes) and zero runtime overhead beyond the bitmap write per branch edge (negligible for network-bound workloads).

D. Case Study: Null Reference Crash in nopCommerce

Among the 14 HTTP 500 responses found in nopCommerce, 6 are distinct null reference exceptions in the product catalog controller triggered by a specific combination of a valid minimum-length string (exactly 3 characters) and a negative integer value for the price field. Both constraints were recovered by SSE:

```
[MinLength(3)]
public string ProductName { get; set; }

RuleFor(x => x.Price)
    .GreaterThan(0);
```

SSE synthesized the boundary values "abc" (length 3) and -1 (below the `GreaterThan(0)` bound). When combined in the same request, the application's price formatting code dereferences a null `CurrencySettings` object. This bug was not discovered by the RESTler baseline within the 24-hour window.

IX. DISCUSSION

Limitations. SSE's regex-based C# parsing does not handle validation constraints implemented in imperative code (e.g., `if (value < 0) throw ...`) or computed at runtime from database lookups. Migrating to Roslyn-based AST analysis would increase precision at the cost of requiring a .NET SDK in the Python pipeline; we consider this a worthwhile

future direction. Additionally, our evaluation targets open-source .NET applications; results on closed-source or polyglot microservice systems may differ.

Applicability. While the current implementation targets .NET, the architectural principle of SSE is language-agnostic: similar static passes could recover Jakarta Bean Validation constraints from Java source, Pydantic models from Python, or Joi schemas from TypeScript. Extending UpsideFuzz to these ecosystems is a natural area for future work.

Ethical Considerations. All experiments were conducted on self-hosted instances of open-source software in isolated Docker environments. No production systems were tested.

X. CONCLUSION

We presented UpsideFuzz, a greybox REST API fuzzing framework for .NET that closes the semantic gap between specification-based grammar generation and application-level validation logic. By statically extracting C# validation constraints and synthesizing boundary-focused dictionary values, UpsideFuzz enables the fuzzer to bypass deep input validation and reach unexplored code paths. Combined with AFL-style edge-coverage feedback collected from SharpFuzz-instrumented .NET processes via shared memory, the Void execution engine evolves a seed corpus that achieves significantly broader coverage than black-box specification-based tools. We hope this work motivates further research into source-aware fuzzing for statically typed, framework-rich server-side ecosystems.

ACKNOWLEDGMENTS

The authors thank the RESTler team at Microsoft Research for making RESTler publicly available, and Nemanja Mijailovic for SharpFuzz.

REFERENCES

- [1] V. Atlidakis, P. Godefroid, and M. Polishchuk, "RESTler: Stateful REST API Fuzzing," in *Proc. 41st IEEE/ACM Int. Conf. on Software Engineering (ICSE)*, 2019.
- [2] P. Godefroid, B.-Y. Huang, and M. Polishchuk, "Intelligent REST API Data Fuzzing," in *Proc. 28th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, 2020.
- [3] N. Mijailovic, "SharpFuzz: Bringing the Power of afl-fuzz to .NET Platform," arXiv preprint arXiv:1905.11645, 2019.
- [4] M. Zalewski, "American Fuzzy Lop (AFL)," <https://lcamtuf.coredump.cx/afl/>, 2013–2017.
- [5] A. Arcuri, "EvoMaster: Evolutionary Multi-context Automated System Test Generation," in *Proc. 11th IEEE Conf. on Software Testing, Validation and Verification (ICST)*, 2018.
- [6] B. Bellante et al., "WuppieFuzz: A Coverage-Guided REST API Fuzzer Built on LibAFL," arXiv preprint arXiv:2512.15554, 2025.
- [7] Apiary, "Dredd: HTTP API Validation Tool," <https://github.com/apiaryio/dredd>, 2013.
- [8] APiFuzzer, <https://github.com/TNO-S3/WuppieFuzz>, 2020.
- [9] D. Tsoy et al., "Schemathesis: An API Testing Tool Based on Hypothesis and Property-Based Testing," <https://github.com/schemathesis/schemathesis>, 2020.